

Gensyn, Delphi LMSR Markets with Automated Settlement

Security Assessment

September 17, 2026

Prepared for:
Harry Grieve
Gensyn

Prepared by: **Guillermo Larregay and Marcelo Morales**

Table of Contents

Table of Contents 1
Project Summary 2
Executive Summary 3
Project Goals 6
Project Targets 7
Project Coverage 9
Automated Testing 11
Codebase Maturity Evaluation 12
Summary of Findings 16
Detailed Findings 17
1. Failed and expired markets can liquidate shares above their purchase cost 17
2. Post-settlement price view functions can report incorrect values for shares 20
A. Vulnerability Categories 23
B. Code Maturity Categories 25
C. Code Quality Recommendations 27
D. Test Suite Coverage Analysis 29
E. Fix Review Results 34
Detailed Fix Review Results 35
F. Fix Review Status Categories 36
About Trail of Bits 37
Notices and Remarks 38

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Guillermo Larregay, Consultant
guillermo.larregay@trailofbits.com

Marcelo Morales, Consultant
marcelo.morales@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
July 21, 2026	Pre-project kickoff call
July 27, 2026	Status update meeting #1
August 3, 2026	Status update meeting #2
August 13, 2026	Delivery of report draft
August 13, 2026	Report readout meeting
August 18, 2026	Completion of fix review
September 17, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Gensyn engaged Trail of Bits to review the security of its Delphi LMSR markets with automated settlement.

Delphi is a collateralized, multi-outcome prediction market protocol that uses a logarithmic market scoring rule (LMSR) to price outcome shares and provide continuous liquidity. Market creators configure the possible outcomes, liquidity parameter, trading fee, and lifecycle deadlines, and they supply the initial collateral that funds the market maker's bounded loss. Traders purchase and sell ERC-6909 shares through a gateway, while the LMSR pricing library adjusts each outcome's implied probability according to the outstanding share supplies. After successful settlement, holders redeem winning shares for collateral; if a market fails or expires, holders liquidate their shares at the market's frozen closing probabilities.

The on-chain system consists of a factory that deploys non-upgradeable market clones, a gateway that exposes trading and lifecycle operations, individual market contracts that custody collateral and maintain share and pricing state, and an upgradeable oracle relayer that connects the gateway to the Truebit WatchTower. Once trading closes, any account can request resolution through the gateway. The relayer submits the market information to the external settlement process and processes the resulting callback by either recording a winning outcome or marking the market as failed.

A team of two consultants conducted the review from July 22 to August 11, 2026, for a total of two engineer-weeks of effort. Our testing efforts focused on the correctness and solvency of the LMSR pricing and accounting model, adversarial trading and terminal payout behavior, the market lifecycle and automated-settlement process, and the access-control, oracle, collateral-token, and ERC-6909 integration boundaries. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

The most consequential issue (TOB-GEN-LMSR-1) arose from the interaction between automated settlement and failed-market liquidation. Any account can request resolution immediately after trading closes, even when the underlying event has not concluded; an indeterminate oracle response can then irreversibly fail the market. Because failed and expired markets liquidate shares at their final marginal prices rather than their average acquisition costs, a trader who anticipates or triggers this condition can extract value from the creator's liquidity subsidy and from the other holders' positions. We also identified an integration risk in the post-settlement price `view` functions (TOB-GEN-LMSR-2). These functions continue to report the frozen LMSR marginal prices after settlement, even though

losing shares have no redemption value. An indexer or other integrator that interprets these values without first checking the market status and winning outcome could misrepresent users' positions.

The codebase demonstrates several strengths. The contracts separate market creation, trading, pricing, custody, and oracle responsibilities; consistently enforce the principal authorization boundaries; and apply conservative rounding throughout. The extensive unit, fuzz, and invariant test suites provide a strong testing foundation, while our differential and mutation-testing campaigns confirmed the core arithmetic behavior and identified targeted opportunities to improve regression coverage.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Gensyn take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Strengthen the automated-settlement trust boundary and its operational controls.** Define and enforce the relationship between trading closure, event finality, and resolution eligibility; authenticate all resolution inputs; provide recovery paths for failed or interrupted requests; and protect relayer changes and upgrades with documented, delayed, and threshold-controlled administrative procedures.
- **Integrate continuous security validation and monitoring into the development lifecycle.** Run invariant, differential, and mutation testing campaigns to improve test coverage, as recommended in appendix D. Add adversarial tests for lifecycle and transaction-ordering risks, enforce coverage thresholds, and establish monitoring and incident-response procedures for settlement, failure, fee-transfer, and administrative events.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	1
Medium	0
Low	0
Informational	1
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Data Validation	2

Project Goals

The engagement was scoped to provide a security assessment of the Gensyn Delphi LMSR markets with automated settlement. Specifically, we sought to answer the following non-exhaustive list of questions:

- Does the fixed-point LMSR implementation preserve the intended pricing formulas, rounding behavior, and approximation bounds across all supported market configurations?
- Does the protocol remain solvent, and does its accounting conserve collateral throughout trading, settlement, redemption, liquidation, fee distribution, and sweeping?
- Can traders exploit pricing, transaction ordering, terminal-state payouts, minimum trade sizes, or exponent limits to extract unintended value or disrupt other users?
- Does the market lifecycle enforce the intended trading, resolution, settlement, failure, and expiration transitions at every timestamp boundary?
- Can an unprivileged actor prematurely resolve a market, force it into a failed or expired state, manipulate its winning outcome, or prevent a valid settlement?
- Does the automated-settlement mechanism securely authenticate, bind, process, and invalidate oracle requests and callbacks, including delayed, duplicated, replayed, malformed, or reverting responses?
- Do initialization, factory registration, access control, ownership transfer, relayer replacement, and upgrade mechanisms appropriately protect privileged operations?
- Do the documentation and test suite accurately represent the protocol's security properties, arithmetic assumptions, lifecycle invariants, and integration requirements?

Project Targets

The engagement involved reviewing and testing the following target.

private-smart-contracts

Repository	https://github.com/gensyn-ai/private-smart-contracts/
Version	Commit `4e312c176cb822d78a825b753cc647ac434298a2`
Type	Solidity
Platform	EVM

The following is the public mirror of the private repository, provided by Gensyn at the conclusion of the engagement:

Public mirror (post-audit tag)

Repository	https://github.com/gensyn-ai/delphi-smart-contracts/
Version	Commit `da49822a9d6cd2c6ad932f00dd50bab40fff05b6`
Type	Solidity
Platform	EVM

The following are the SHA-256 hashes of the in-scope files:

```
src/factory/DelphiFactory.sol
              f5772324575e19103dfb96b7319265cec7a10971bcfb4f8194dbeba12d05c0a6
src/factory/IDelphiFactory.sol
              b93aefa3e8faa53655130aec839f023d00c016ae5cfe5145315ba18404788ea2
src/factory/IDelphiFactoryErrors.sol
              7786604f182455d243d7fbae12fe807461196e0515e7554501b33e1601b92ab0
src/IDelphiMarket.sol
              46d23f9c0ddadcc7e585eb05f8480a659b933bf4d4fbc6b4a8723b271b4c2473
src/IErrors.sol
              a0e26e30ade6759e95f4cbfd05e9de59c7f4d9b56066512e7fad8ca7db729547
src/IOracle.sol
              facb56045c441fe5c9aaf7ac30fca312566e5afca4f694de51e928e3759c7739
src/lmsr/gateway/ILmsrGateway.sol
```

d76c03f5b32ef38dd2f4339f568a937a6bad10b5527b53a73b73bdc2511d13bc src/lmsr/gateway/ILmsrGatewayErrors.sol

d8c62391e48017d7722b7c5d263bf8f8174d1e375c6f4702d6b6cb4c9c66eb8d src/lmsr/gateway/LmsrGateway.sol

5d8bec5b4b27fcdaf820aeb2617acd1f3080c8564659183816d4cdebd0a89273 src/lmsr/implementation/ILmsrMarket.sol

37014c193268e0caa79b66810a77f69f83c01c419834baf75804766bc2c85e41 src/lmsr/implementation/ILmsrMarketErrors.sol

fede822e252ab1ac349e639c7df4e785de23a5fa1b5cacc2a8b63050c58086e5 src/lmsr/implementation/ILmsrMarketTypes.sol

a69729b97188624f4ccc1dcbb36759cd788589e69786b99ba2771ac8fabbe52c src/lmsr/implementation/LmsrMarket.sol

0d9a009ecc5fb4eb8897c5a139c250c45c252c61cbe9b8962529089b6110501a src/lmsr/math/ILmsrMathErrors.sol

dedcda6fec88ad05cf52bb5ab158cb4d56c24d1314c8b7cf71504e92c8756b25 src/lmsr/math/LmsrMath.sol

21f104277c0d08792212e69e5b6500d21f7a9c65772e2ead5979096e71eb8499 src/oracle/truebit/abstract/Types.sol

56001280c9553de38d09e555466da1a432ff1a9ea37895a0312eae4540c63c28 src/oracle/truebit/interfaces/IBaseTBContract.sol

3d95330021051697141141178 01a5acdc2742ed459df7d9153300756190b8bad src/oracle/truebit/interfaces/IWatchTower.sol

3fee9d5e2eda96dfdef8fe9cd08c014b86eb3d5c2bff77b2685d68c759e08fbf src/oracle/TruebitOracleRelayer.sol

53235c5aea2ead61f898a4bd5b023e33a9763139dbbc7b2bb06ee6adb990a943

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Manual review of the LMSR pricing implementation.** This included a review of the implementation's correspondence with the academic model, fixed-point arithmetic, approximation bounds, rounding behavior, and parameter limits. We also analyzed collateral accounting, fee calculations, solvency guarantees, and terminal payouts.
- **Review of the market lifecycle and automated-settlement process.** This included a review of timestamp boundaries, settlement locks, oracle requests and callbacks, failure handling, access controls, relayer replacement, and upgradeability. We supplemented this review with read-only analysis of deployed markets and failed-market transactions on the Gensyn testnet.
- **Review of the system's ERC-6909 compliance and asset flows involving outcome shares and collateral tokens.** This analysis covered issuance, transfers, approvals, purchases, sales, redemptions, liquidations, fee distribution, donations, and potential asset-stranding conditions.
- **Adversarial economic analysis.** This analysis covered the system's vulnerability to arbitrage, price and outcome manipulation, failed-market liquidation, transaction ordering, sandwich attacks, flash loans, denial-of-service strategies, and interactions between these techniques.
- **Review of the test suites.** We executed and assessed the existing unit, fuzz, and invariant suites and reviewed the fork-test implementation. We also developed and executed proof-of-concept tests, negative controls, high-precision differential tests, and Echidna and Medusa property-based fuzzing campaigns.
- **Mutation testing.** We implemented a mutation-testing campaign across the in-scope contracts and analyzed its results, including manual classification of surviving mutants and identification of systematic test gaps. We complemented this work with static analysis, specification comparisons, and documentation review.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The review primarily covered the `LmsrMath`, `LmsrGateway`, `LmsrMarket`, `TruebitOracleRelayer`, and `DelphiFactory` contracts. We reviewed interfaces,

tests, configuration, and deployment scripts as supporting evidence. The third-party libraries and dependencies were out of scope.

- The off-chain automated-settlement implementation was not in scope. Therefore, our analysis covered the on-chain oracle boundary but could not verify metadata retrieval, prompt construction, model execution, caching, result validation, or protections against malicious market content.
- The Truebit WatchTower implementation, collateral-token internals, indexer, and user interface were also out of scope. We analyzed their on-chain integration points and trust assumptions but did not assess their internal security.
- The mutation-testing campaign used the Foundry unit test suite and did not include the invariant or fork suites. We reviewed the fork-test implementation but did not execute its RPC-dependent tests.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
mewt	Mewt is a mutation testing tool. Mutation testing works by making small changes to source code and then running the test suite against each change.	Appendix D

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The protocol is arithmetic-heavy and uses Solidity 0.8.30. A stateless `LmsrMath` library implements the LMSR pricing and trade-validation formulas. The library rounds user-facing purchase costs up and sale proceeds down, annotates the intended direction of economically relevant rounding operations, and applies intended conservative margins to the exponential and logarithm approximations. The core market parameters have documented minimum and maximum values, and the Foundry fuzz and invariant suites exercise the pricing formulas and accounting properties. The test suite can be improved by comparing the implementation against a high-precision reference model across the supported input domain. This differential testing would measure the actual approximation error and demonstrate whether the selected error margins conservatively bound it.	**Satisfactory**
Auditing	Events cover market creation, trading, settlement, failure, redemption, liquidation, fee transfers, and relayer configuration. Gateway and relayer events index the market proxy where per-market filtering is required, and the project guide documents the event ABI as a stable integration surface for off-chain consumers. At the time of the audit, the client had not provided an incident-response plan or evidence of an off-chain monitoring and alerting process. Documenting how operators consume critical events, investigate anomalous activity, and respond to failures would improve this category.	**Moderate**
Authentication /	The contracts separate privileges across the market,	**Moderate**

Access Controls	gateway, and oracle relayer. The market restricts its LMSR lifecycle and accounting entry points to the gateway. The gateway restricts settlement and failure callbacks to the configured oracle relayer, and the relayer restricts oracle callbacks to the WatchTower. The gateway and relayer use two-step ownership, initializers are one-shot, the gateway accepts only factory-registered markets, and unit tests cover unauthorized calls to privileged functions. The code does not require different accounts to hold the privileged roles, nor does it require owner accounts to be contracts rather than EOAs. The repository includes an optional timelock deployment script, but the deployment flow does not automatically transfer gateway or relayer ownership to that timelock. Therefore, the production role assignments and ownership protections depend on deployment procedures that were not available in the audited scope.	
Complexity Management	The codebase separates the factory, gateway, market implementation, pricing library, and oracle relayer into components with distinct responsibilities. Interfaces and source-contract custom errors are defined separately, the pricing library is stateless, and named constants describe critical bounds and fixed-point values. The project also centralizes the market lifecycle in the `marketStatus` state-machine function. Several quote, settlement, liquidation, and oracle-callback functions remain comparatively long because they combine validation, accounting, and external interactions. Adding a complexity or static-analysis check to continuous integration would help identify future increases in cyclomatic complexity, nesting, and duplication.	**Satisfactory**
Cryptography and Key Management	The in-scope contracts implement no custom security-critical cryptography or key management.	**Not Applicable**
Decentralization	Markets are non-upgradeable EIP-1167 clones of the implementation selected by their factory, and their lifecycle parameters cannot be changed after initialization. The market and gateway expose no	**Moderate**

	privileged function that can directly withdraw user collateral. The Truebit oracle relayer is UUPS-upgradeable by its owner. A separately configured gateway owner can also replace the relayer through `setOracleRelayer` without an on-chain delay or a user exit period. The repository provides an optional timelock deployment, but it does not automatically assign either ownership role to the timelock. Consequently, the settlement path depends on the production ownership configuration, which was not available in the audited scope.	
Documentation	The project provides documentation in the repository and on Gensyn's public website. The repository guides describe the architecture, the on-chain-to-off-chain integration surface, the bounds on the core market parameters, and a dedicated set of invariants and risky assumptions. NatSpec comments document the public and external functions implemented in the application contracts, while inline comments explain rounding direction, checks-effects-interactions decisions, and lifecycle transitions. Gensyn's public LMSR article supplements the repository documentation by presenting the cost, price, trade-cost, and worst-case-loss formulas and linking to Robin Hanson's 2002 paper, "Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation." The external material provides the theoretical basis for LMSR, but the repository does not contain a project-specific specification that maps the published model to Delphi's fixed-point arithmetic, approximation margins, rounding rules, fees, and terminal-state behavior. The repository also lacks architecture and data-flow diagrams. Adding these diagrams, linking the external references directly from the repository, and documenting the correspondence between the theoretical formulas, contract implementation, and executable invariants would improve the documentation.	**Moderate**
Low-Level Manipulation	The oracle relayer uses assembly only for the upgradeable storage slot. The project contains no low-level operations or calls.	**Not Applicable**

Testing and Verification	Testing is a primary component of the codebase. The suite includes unit tests, a stateful invariant suite with end-to-end and convergence-oriented handlers, and fork tests against deployed USDC contracts. Foundry configures ordinary fuzz tests and invariant campaigns, with invariant reverts treated as failures. Continuous integration is configured to build the Delphi project and run tests on the main branch. Appendix D provides guidance for improving the existing test suite and reducing regressions through mutation testing. Integrating an independent property-based harness, such as Echidna or Medusa, into the project's continuous integration workflow would provide additional coverage. Formal verification of precisely specified arithmetic and accounting properties would further increase assurance that the implementation satisfies those properties.	**Moderate**
Transaction Ordering	Trades provide slippage bounds both for purchases and sales. The permit-based purchase path forwards a deadline to the collateral token's permit function. The invariant suite tests buy-and-sell round-trip non-profitability and liquidation order independence, but these properties do not cover adversarial transaction ordering. The LMSR cost function constrains the aggregate cost of trades that reach the same final state, but it does not prevent an attacker from sandwiching or temporarily moving the market against another user's transaction. Slippage bounds limit adverse execution but can instead cause the affected transaction to revert. The documentation should centralize these ordering and MEV risks, and dedicated adversarial tests should exercise front-running, sandwiching, and keeper-fee races.	**Moderate**

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Failed and expired markets can liquidate shares above their purchase cost	Data Validation	High
2	Post-settlement price view functions can report incorrect values for shares	Data Validation	Informational

Detailed Findings

1. Failed and expired markets can liquidate shares above their purchase cost

Severity: **High**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-GEN-LMSR-1
Target: `delphi/src/lmsr/implementation/LmsrMarket.sol`, `delphi/src/lmsr/math/LmsrMath.sol`	

Description

When a market reaches the `FAILED` or `EXPIRED` terminal state, holders exit through liquidation, which pays each position out at the outcome's final LMSR marginal price. Because buying an outcome pushes its marginal price up along the LMSR curve, that final price is strictly higher than the average price paid to open the position, so a liquidation returns more collateral than the position cost. Therefore, a trader who anticipates a terminal state can inflate an outcome's price immediately beforehand and liquidate the inflated position for a profit drawn from the market pool, at the expense of the creator's subsidy and of the other holders.

A buy operation charges the increase in the LMSR cost function, `b * ln(newExpSum / currentExpSum)`, which is the area under the rising marginal-price curve. Therefore, every share except the last costs less than the final marginal price, so the position's average cost is strictly below it.

Liquidation is reachable only in the two terminal states, as shown in figure 1.1.

```
// Checks: Only valid in EXPIRED or FAILED status
MarketStatus _status = marketStatus();
if (_status != MarketStatus.EXPIRED && _status != MarketStatus.FAILED) {
    revert MarketNotExpiredNorFailed(_status);
}
```

Figure 1.1: Liquidation is available only in the expired and failed terminal states. (private-smart-contracts/delphi/src/lmsr/implementation/LmsrMarket.sol#L442–L446)

The reward for each liquidated position is the position's share count multiplied by the outcome's final marginal price, so the price of the last and most expensive share is applied to the entire position (figure 1.2).

```
/* Note:
 * totalTokensOut = (outcome1SharesIn * outcome1SpotPrice) + ... + (outcomeNSharesIn
* outcomeNSpotPrice)
 * totalTokensOut = (outcome1SharesIn * outcome1Exp / expSum) + ... +
(outcomeNSharesIn * outcomeNExp / expSum)
 * totalTokensOut = ((outcome1SharesIn * outcome1Exp) + ... + (outcomeNSharesIn *
outcomeNExp)) / expSum
 */
function liquidatorTotalReward(uint256 numeratorSum36, uint256 currentExpSum,
uint256 tokenDecimalScaler)
    internal
    pure
    returns (uint256)
{
    return numeratorSum36 / (currentExpSum * tokenDecimalScaler); // Note: Down
(against user)
}
```

Figure 1.2: The terminal reward values each position at its final marginal price. (private-smart-contracts/delphi/src/lmsr/math/LmsrMath.sol#L237–L248)

To keep liquidations order-independent, the protocol freezes the pricing state at the terminal moment: it pulls liquidated shares into the market instead of burning them, leaving the outcome supplies and the cached `expSum`—and therefore the final marginal price—unchanged. The inflated price consequently applies to every liquidator. The difference between this final-price payout and the integrated purchase cost is funded from the market creator's deposit and other traders' purchases, and the pool remains solvent; the impact is a reallocation to the trader rather than a pool deficit. Trading fees narrow the margin but do not remove it for sufficiently concentrated purchases.

Exploitation still requires the attacker to establish the manipulated position before the trading deadline because purchases are accepted only while the market is `OPEN`. However, the attacker does not always need to predict an independent or unlikely settlement failure. If the market's resolution criteria depend on information that will become available only after the trading deadline, the attacker can identify this timing mismatch from the market prompt and timestamps, trade before the market closes, and call the permissionless `resolveMarket` function immediately afterward. The oracle may then return an unknown or invalid outcome because the relevant event or evidence window has not concluded, causing the market to enter the `FAILED` state and liquidate positions at the attacker-influenced closing prices. Therefore, unlike `EXPIRED`, which is reached only after the settlement deadline, `FAILED` can be triggered near the beginning of the settlement window.

For markets whose trading and evidence windows are properly aligned, exploitation still requires advance knowledge of a genuine non-settlement condition; for misaligned markets, the immutable prompt and deadlines may provide the attacker with that knowledge before trading closes.

Exploit Scenario

Mallory, an attacker, creates a market that is prone to settlement failure; for example, it requires a piece of information that will be available a day after the trading window closes. The market is a three-outcome market with a liquidity parameter of 100, the maximum 5% trading fee, and prices of roughly 95%, 3%, and 2%. While the market is still `OPEN`, and right before the trading deadline, Mallory buys approximately 169.46 shares of the 2% outcome, paying about 8.96 collateral tokens, including the fee, and raising that outcome's price to 10%. The trading window ends, and the attacker immediately calls `resolveMarket` to make it fail. Once the settlement fails, Mallory calls `liquidate` on the position. The protocol values all 169.46 shares at the final 10% marginal price and pays out about 16.95 collateral tokens. She nets roughly 7.98 collateral tokens, an approximately 89% return on the collateral committed, drawn from the pool at the expense of the creator's subsidy and of the holders of the other outcomes, whose shares now liquidate at the prices her purchase depressed, and without bearing the outcome risk of a normal settlement.

Recommendations

Short term, introduce a configurable delay between the end of trading and the beginning of resolution. Prevent resolution requests until the underlying event has concluded and authoritative evidence should be available, including an appropriate publication buffer. Preserve sufficient time afterward for oracle execution, and treat responses indicating that evidence is not yet available as retryable rather than immediately failing the market.

Long term, redesign failed- and expired-market payouts so they do not apply the final marginal price retroactively to every share in a position. Use a cancellation mechanism that cannot be increased through pre-failure price manipulation, and formally distinguish the trading period, evidence-finality period, resolution period, and final settlement deadline. Provide retry, fallback-oracle, or dispute paths before cancellation, and test adversarial scenarios involving manipulated closing prices and premature resolution requests.

2. Post-settlement price view functions can report incorrect values for shares

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-GEN-LMSR-2
Target: `delphi/src/lmsr/implementation/LmsrMarket.sol`	

Description

After a market settles, `spotPrice` and `spotImpliedProbability` continue to return the frozen LMSR marginal price for every outcome. Therefore, a losing share can report a nonzero price even though its claim value is zero. An off-chain consumer that treats values returned by these functions as current economic values without first checking the market status and winning outcome could overvalue losing shares.

The settlement process calculates the collateral required to pay all winning shares, assigns the remaining collateral to `losingPayout`, records the winning outcome, and reduces the market pool to `winningPayout`. It does not update the cached exponent sum or any outcome supply. The price `view` functions do not inspect `marketStatus` or `winningOutcomeIdx` (figure 2.1). Instead, they continue to derive values from the unchanged total supplies and cached exponent sum. The batched variants inherit the same behavior.

```
/// @inheritdoc ILmsrMarket
function spotPrice(uint256 outcomeIdx) public view validOutcomeIdx(outcomeIdx)
returns (uint256) {
    return _market.config.b
        .spotPrice({
            outcomeSupply: totalSupply(outcomeIdx),
            marketExp: _market.expSum,
            tokenDecimalScaler: TOKEN_DECIMAL_SCALER
        });
}

/// @inheritdoc ILmsrMarket
// Note: Do not use the math library directly (or the validOutcomeIdx check will be
bypassed)
function spotPrices(uint256[] calldata outcomeIndices) external view returns
(uint256[] memory prices) {
    prices = new uint256[](outcomeIndices.length);
    for (uint256 i = 0; i < outcomeIndices.length; i++) {
        prices[i] = spotPrice(outcomeIndices[i]);
    }
}
```

```
/// @inheritdoc ILmsrMarket
function spotImpliedProbability(uint256 outcomeIdx) public view
validOutcomeIdx(outcomeIdx) returns (uint256) {
    return
        _market.config.b.spotImpliedProbability({outcomeSupply:
totalSupply(outcomeIdx), marketExp: _market.expSum});
}
```

Figure 2.1: Price `view` functions derive values from pricing state without considering settlement. (private-smart-contracts/delphi/src/lmsr/implementation/LmsrMarket.sol#L642–L665)

In contrast, the `redeem` function considers only the caller's winning-outcome balance (figure 2.2). A holder that owns losing shares but no winning shares cannot redeem those losing shares and receives no collateral for them.

```
function redeem(address redeemer)
    external
    nonReentrant
    onlyGateway
    ifStatus(MarketStatus.SETTLED)
    returns (uint256 winningSharesIn, uint256 tokensOut)
{
    // Get redeemer winning shares
    winningSharesIn = balanceOf(redeemer, _market.winningOutcomeIdx);

    /*
     * tokensOut > 0
     * winningSharesIn / TOKEN_DECIMAL_SCALER > 0
     * winningSharesIn >= TOKEN_DECIMAL_SCALER
     */
    if (winningSharesIn < TOKEN_DECIMAL_SCALER) {
        revert WinningSharesInAreLessThanTokenDecimalScaler(winningSharesIn,
TOKEN_DECIMAL_SCALER);
    }

    // Calculate tokens out
    tokensOut = winningSharesIn.redeemerReward({tokenDecimalScaler:
TOKEN_DECIMAL_SCALER});
```

Figure 2.2: Redemption derives value exclusively from winning shares. (private-smart-contracts/delphi/src/lmsr/implementation/LmsrMarket.sol#L398–L418)

No state-changing contract in the reviewed repository consumes these price `view` functions after settlement. However, the UI, indexer, marketplace, third-party integrations, and other consumer implementations were out of scope for this review, so we could not determine whether a supported integration interprets the values incorrectly.

Recommendations

Short term, document that `spotPrice` and `spotImpliedProbability` represent LMSR marginal or closing values rather than terminal claim values, and require consumers to interpret them together with `marketStatus` and `winningOutcomeIdx`.

Long term, expose a separate status-aware claim-value `view` function that returns one collateral token per whole winning share and zero for losing shares after successful settlement.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Have ERC-6909 getters return zero for unconfigured outcome IDs, or document the restricted behavior.** `balanceOf` and `totalSupply` revert for IDs outside the configured outcome range, while approvals, allowances, and zero-value transfers accept the same IDs. ERC-6909 does not explicitly prohibit these reverts, but having the getters return zero would align them with the EIP reference implementation, OpenZeppelin's behavioral tests, and common integration expectations.
- **Remove `outcomeSuppliesSum` from storage unless external consumers require it.** The value is updated on every mint and burn operation but is not used by liquidation or any other internal calculation, causing unnecessary storage writes. Remove it or calculate it on demand; if integrations require the getter, document it.
- **Update the documentation to include failed markets.** Several comments describe liquidation as available only for expired markets, while the implementation also permits liquidation when a market is explicitly `FAILED`.
- **Clarify the meaning of `keeperFeePaid` and `oracleFeePaid`.** These flags become true when fees are paid, when zero fees are processed, or when unpaid fees are returned or accrued to the creator. Rename them to represent finalized fee accounting or document every state represented by true.
- **Correct the documentation for `initialDeposit_`.** The interface says that this parameter must cover the market's maximum loss, but initialization actually checks the clone's complete token balance. Document this behavior or explicitly validate the transferred parameter if prefunding should not count.
- **Document the WatchTower's treatment of zero-value code hashes for registered tasks.** The relayer submits registered API tasks with `codeHash` set to `bytes32(0)`, relying on external confirmation that this field is ignored. Implement an integration test so future WatchTower changes cannot silently invalidate this assumption.
- **Remove the unreachable lower-bound check for `tradingFeesRecipientPct`.** The value is unsigned and its configured minimum is zero, so it can never be below the minimum. Removing the branch and its unused error would simplify the validation logic.

- **Use named fixed-point constants consistently throughout `LmsrMath`.** The library defines `_ONE` but there are several instances of raw `1e18` and composite scaling literals. Reusing named constants would make units clearer and reduce the risk of inconsistent scaling during future changes.

D. Test Suite Coverage Analysis

Line coverage measures the lines of code that were tested; it does not indicate whether an incorrectly implemented line would cause a test to fail. Mutation testing measures the latter. A mutation test makes many small, deliberate edits, called mutants, to the source (such as changing > to >=, deleting a validation check, or flipping + to −), and then runs the whole test suite against each mutant and records the outcome:

- A mutant is **caught** if at least one test fails. The suite notices the change.
- A mutant is **uncaught (it survived)** if every test passes. Nothing in the suite detects the change. This is a genuine gap, unless the edit happens to be equivalent (for example, replacing `x == 0` with `x <= 0` for unsigned types).
- A mutant is **skipped** if it is not tested because a more severe mutant on the same line already survived.

The mutation campaign resulted in 2,005 mutants. Of the 1,667 actually tested, 1,494 were caught and 173 survived (65 equivalent, 108 real gaps). The remaining 338 were skipped. Excluding equivalents, the suite catches 93.3% of the mutants it tested. This is an overall strong result, with the gaps concentrated in a few recurring patterns.

Coverage Gaps

The gaps fall into two groups. The first group is a set of weaknesses in the shared test helpers, each of which hides mutants across several files. The second group is the set of gaps specific to individual contracts.

Shared-Helper Weaknesses

- **Tests that compute their expected answer from the code being tested.** Several trade tests calculate expected values by calling the same production functions whose behavior they are meant to verify. When a mutant changes one of these functions, the actual and expected values change together, and the comparison still passes. This affects the nonzero-fee sweep test and the buy/sell validity helpers. The settlement tests have a separate weakness: they settle the market without prior trading, so every fee-related value is zero. Expected values should instead come from fixed examples, independently written formulas, or an independent reference implementation.
- **Tests that accept any error instead of the right one.** The sell tests catch reverts and pass their selectors to `_quoteSellExactInAllowedErrors`, which accepts any of four sell-path errors. Because no test specifies which error a given input should produce, mutations that make every sell revert, swap one rejection reason for another, or cause the operation to fail later for an unrelated reason all pass

unnoticed. Each rejection test should construct a specific input and assert its exact expected error.

- **Tests that throw away inputs they should be checking.** The buy helper `_quoteBuyOrSkip` discards dust-sized buys instead of asserting that such buys are rejected at the minimum-trade-size floor. As a result, the buy-side minimum is never verified. A value the contract is supposed to reject should be considered a test case.
- **Test setups with a single participant.** The redeem and liquidate tests use one account that holds the entire relevant position. That makes the pool balance exactly equal to that account's payout, so a mutation that corrupts the pool bookkeeping (for example, turning a subtraction into a modulo) happens to produce the same zero result and survives. These paths need a setup with more than one holder, where the pool is larger than any single payout, so the bookkeeping is actually exercised.
- **State that is changed but never read back.** The market tracks the sum of all per-outcome total supplies (`outcomeSuppliesSum`, updated on every mint and burn), but no in-scope test ever reads it, so four different corruptions of that update logic survive. More broadly, the suite lacks a strict accounting check after each operation. In controlled tests that exclude unsolicited token transfers, the market's token balance should equal the pool plus accrued trading fees, unpaid keeper and oracle fees, and the market creator's claimable balance. Liquidation and redemption payouts should not be added separately because they are already reserved within the pool.

Contract-Specific Gaps

- **`src/oracle/TruebitOracleRelayer.sol`.** This contract receives results from the Truebit WatchTower, so its checks are security boundaries. The check that marks a market as failed when the oracle reports a non-successful computation cannot be verified by the current tests. Every test of a non-success result sends empty result data, and empty data trips a malformed-length check that also fails the market. So deleting the success check entirely changes nothing the tests can see. A non-success result carrying a well-formed outcome, which without the check would be recorded as a successful settlement, is never sent, and the tests inspect only the final market state and part of the emitted event, never the field that says why the market failed. Separately, two constructor checks (rejecting a zero-value oracle-gateway address and a zero-value timeout) have no tests, and the contract's initializer lockdown is untested.
- **`src/lmsr/gateway/LmsrGateway.sol`.** `LmsrGateway` is the only entry point for buying and selling shares. The constructor sets a minimum trade size but no test asserts its value, so a mutation that zeroes it, disabling the minimum, goes unnoticed. When a gasless-approval (`permit`) signature fails, the gateway reads the

user's token allowance to produce a clearer error; but both permit-failure tests use a zero-value allowance, so a mutation that reads the wrong account, or that always reports "insufficient allowance," produces the same result as the original. The sell slippage check has no test at its exact boundary. All read-only pass-through functions have no direct tests. These view-only mutations do not alter on-chain state, but they can make public queries revert and disrupt off-chain consumers.

- **`src/lmsr/implementation/LmsrMarket.sol`.** At settlement, the market splits accrued trading fees between the protocol's fee recipient and the market creator, zeroes the fee tally, and pays out. However, the settlement test performs no prior trading, so there are no fees to split and every mutation of that block reduces to a zero-value no-op. Several lifecycle boundaries are also never exercised exactly: the market's status transitions (open to awaiting settlement to expired) are tested one second past each deadline rather than at the exact deadline, so off-by-one mutations survive. The suite also lacks tests for liquidation of a failed market, the smallest valid redemption, zero-payout liquidation, and the zero-surplus failed/expired sweep. An over-strict mutation of the last check can prevent `failMarket`, `trySweep`, and `liquidate` from completing when the pool exactly equals the reserved liquidation payout. It does not affect successful settlement.

- **`src/lmsr/math/LmsrMath.sol`.** This file has 23 surviving mutants, 17 of which represent real test gaps. Seven financial and accounting survivors involve fee calculations and conservative pricing bounds whose expected results are computed through the same mutated functions. The remaining gaps arise from missing boundary and exact-error tests, missing direct coverage of the price `view` functions, and an untested valid-sell boundary. One surviving fee mutation leaves the sell fee unaccounted, while another directs all accrued fees to the market creator instead of splitting them with the protocol recipient. The pricing helpers compute conservative upper and lower bounds so that rounding favors the protocol over the trader. Flipping a bound's direction can let buyers pay too little or sellers withdraw too much and, in one case, can reduce the safety margin that guarantees the pool can cover all payouts.

- **`src/factory/DelphiFactory.sol`.** The only real surviving mutation gaps identified in this contract are its two untested read-only helpers: one returns a range of deployed market addresses, and the other batch-checks whether a list of addresses are known markets. Unconditional-revert and range-boundary mutations survive because neither function is invoked by the test suite. Both functions are part of the public interface.

Recommendations

These recommendations are ordered by expected return on testing effort. The first group corrects shared expected-value calculation weaknesses, while the following groups add specific missing cases.

Priority 0: Fix the Shared Test Helpers

- **Make expected values independent.** In the settlement, sweep, sell, and buy/sell-validity tests, compute expected fee splits and price bounds from fixed examples, independently written formulas, or a separate reference implementation rather than calling the functions under test. Pair this change with a nonzero-fee settlement setup. Together, these changes will expose the seven financial and accounting math survivors involving fee and bound calculations and the corresponding settlement fee-handling mutations.
- **Assert the specific rejection, not any rejection.** Replace the use of the broad sell-error allowlist with deterministic tests that each expect one specific error for one constructed input. Add one deterministic successful sell operation that verifies the exact amount received. Assert the expected rejection for below-minimum buy operations instead of discarding those inputs.
- **Add accounting-conservation checks to the trade tests.** After every buy, sell, redeem, liquidation, sweep, and settlement operation, assert that the market's token balance equals the pool plus accrued trading fees, unpaid keeper and oracle fees, and the market creator's claimable balance. Also assert that `outcomeSuppliesSum` equals the sum of the per-outcome total supplies. These assertions should run in controlled tests that exclude unsolicited token transfers.

Priority 1: Security-Boundary and High-Value Cases

- **Add oracle security checks.** Test a non-success oracle result that carries a valid outcome, and assert that the market enters `FAILED` rather than `SETTLED` with the correct failure reason. Inspect the event's data rather than only its indexed fields. Add the two missing constructor rejection tests for a zero-value gateway and a zero-value timeout. Add a test confirming that the implementation contract cannot be initialized.
- **Test settlement with real fees.** Have the settlement test perform trading first so that fees exist, calculate the split independently, and verify the recipient balance, creator reward, zeroed fee tally, and final market accounting.
- **Test multi-participant bookkeeping.** Create two holders with redeemable or liquidatable positions, process only one holder, and assert that the pool retains the second holder's reserved payout.
- **Implement direct tests for the math functions.** Test the fee functions, conservative pricing bounds, `spotPrice`, and `spotImpliedProbability` against independently computed expected results. For implied probabilities, assert that their sum does not exceed `1e18` and that the difference between `1e18` and the sum does not exceed `outcomeCount - 1 wei`. Each probability calculation rounds down, so the sum can be slightly less than one.

Priority 2: Boundaries, Lifecycle, and Read-Only Functions

- **Add tests for edge cases.** Test trades whose amounts exactly equal the user's stated limits; a sale that reduces an outcome's total supply to zero; the smallest valid redemption; status at each exact deadline; liquidation of a failed market; zero-payout liquidation; and a zero-surplus failed/expired sweep. Add literal assertions for `MIN_TOKENS_DELTA`, `MAX_TRADING_WINDOW`, and `MAX_SETTLEMENT_WINDOW` instead of deriving both the test inputs and expected values from the same contract getters.
- **Add tests for the read-only functions.** Add a factory test suite for the two untested range and batch functions, covering both their rejection cases and their exact returned contents. Add tests for the market and gateway read-only functions that confirm the list-returning versions match their single-value counterparts and reject malformed inputs where applicable. Prioritize `spotPrices`, `spotImpliedProbabilities`, `totalSupplies`, `marketStatus`, and `getMarket` because off-chain indexers consume them.

E. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On August 18, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Gensyn team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

Gensyn resolved TOB-GEN-LMSR-2 and partially resolved TOB-GEN-LMSR-1; for the latter, Gensyn implemented our short-term recommendation to separate trading closure from resolution eligibility, but did not address the difference between purchase cost and terminal payout price, electing to accept that risk. Gensyn also addressed several of the code quality recommendations in appendix C and made progress toward implementing the codebase maturity recommendations, particularly in the documentation and testing and verification categories.

For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Failed and expired markets can liquidate shares above their purchase cost	High	Partially Resolved
2	Post-settlement price view functions can report incorrect values for shares	Informational	Resolved

Detailed Fix Review Results

TOB-GEN-LMSR-1: Failed and expired markets can liquidate shares above their purchase cost

Partially resolved in PR #31. The fix implements the short-term recommendation to separate trading close from resolution eligibility, removing the attacker's ability to fail a market in the block right after the trading deadline. The pricing issue itself is unchanged, and liquidation still values every share at its outcome's frozen final marginal price. Markets that genuinely fail, naturally expire, or carry a resolve delay shorter than evidence finality remain exploitable. The delay is chosen by the market creator, who can be malicious and may pick the 15-minute floor, allowing resolution to be requested as early as the protocol permits even if the data required for the execution is not yet available. The Gensyn team stated that it will consider implementing off-chain alternatives to minimize the occurrences of market failures, such as adding a retrying ability in the backend; however, we could not verify whether any mitigations have been implemented, as those are out of scope for the audit, and the fix-review PR contains no contract-side changes supporting them.

TOB-GEN-LMSR-2: Post-settlement price view functions can report incorrect values for shares

Resolved in PR #31. `spotPrice` and `spotImpliedProbability` now branch on `marketStatus` and return claim values once a market is settled: one whole collateral token per whole share and probability 1 for the winning outcome, and 0 for every losing outcome. Expired and failed markets report the frozen LMSR closing prices, exactly what liquidation pays out. The settled winning value matches `redeemerReward`, so the `view` functions agree with what redemption actually pays.

F. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Gensyn under the terms of the project statement of work and has been made public at Gensyn's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.